UPN

UNIVERSIDAD
PRIVADA
DEL NORTE

Control de Versiones

Semana 1

Agenda

1. Problemática: versionamiento de archivos
2. Control de versiones: fundamentos
3. Introducción a Git: primeros pasos
4. Repositorios y ramas
5. Práctica y cierre, mi primer repo

Método de Evaluación del Curso

Tareas Prácticas

Ejercicios aplicados para reforzar los conceptos vistos en clase y desarrollar habilidades técnicas

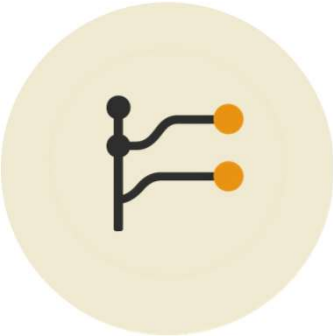
Participación

Intervenciones en clase, trabajo en equipo y contribución al ambiente de aprendizaje.

Evaluaciones Formales

Exámenes y pruebas objetivas para medir el dominio de los contenidos. (T1, T2 y EF)

Objetivos y Aprendizajes esperados



Entender control de versiones

Aprender los conceptos fundamentales del control de versiones para gestionar cambios en proyectos y evitar errores comunes

Manejar comandos básicos de Git

Dominar los comandos esenciales de Git para crear, guardar y recuperar versiones de archivos de manera eficiente.

Colaborar con GitHub

Desarrollar habilidades para trabajar en equipo usando GitHub, facilitando la colaboración y el seguimiento de contribuciones en proyectos.

Cómo manejamos versiones de Archivos?

01 Inicio del Proyecto

Los estudiantes reciben un archivo base para comenzar el trabajo colaborativo

02 Ediciones Simultaneas

Cada estudiante realiza cambios por separado, sin coordinación ni control de versiones.

03 Versiones Desordenadas

Cada estudiante realiza cambios por separado, sin coordinación ni control de versiones.

04 Confusión y Conflictos

Intentan combinar versiones y aparecen conflictos y pérdida de información

05 Frustración

La falta de control frena el avance y eleva la tensión del trabajo en equipo.

06 Reflexión

Se evidencia la necesidad de una herramienta para gestionar versiones.

Solución al versionamiento de archivos

Pérdida de Cambios

Los archivos sobrescritos o eliminados accidentalmente pueden causar la pérdida de trabajo importante.

Conflictos de Colaboración

Varias personas editando el mismo archivo generan versiones incompatibles y confusión.

Organización con Git

Permite guardar versiones, restaurar cambios y evitar pérdidas accidentales.

Resolución de Conflictos con Git

Facilita la integración de cambios de varios colaboradores y resuelve conflictos de manera controlada.

↑

Conceptos Fundamentales del control de versiones

¿Qué es el control de versiones?

Definición

El control de versiones registra los cambios en archivos a lo largo del tiempo, permitiendo volver a estados anteriores y colaborar eficazmente.

Impacto

Según estudios, la adopción de control de versiones reduce errores y mejora la productividad en equipos de desarrollo hasta en un 40% .

Problemas sin el control de versiones

Sobrescribir Archivos

Los miembros del equipo pueden sobrescribir accidentalmente el trabajo de los demás, lo que provoca confusión y la pérdida de actualizaciones importantes.

Trabajo Perdido

Sin copias de seguridad ni historial, el código y la documentación valiosos pueden perderse permanentemente, lo que ralentiza el progreso.

Seguimiento de cambios

Dificultad para identificar quién realizó los cambios y cuándo, lo que dificulta la resolución de problemas o la corrección de errores.

Conflictos de integración

La fusión del trabajo de múltiples colaboradores suele generar conflictos, lo que requiere tiempo y esfuerzos adicionales para su resolución.

Beneficios: historial, colaboración y reversión

Historial Completo

Rastrea cada cambio del código para auditoría, transparencia y aprendizaje del equipo.

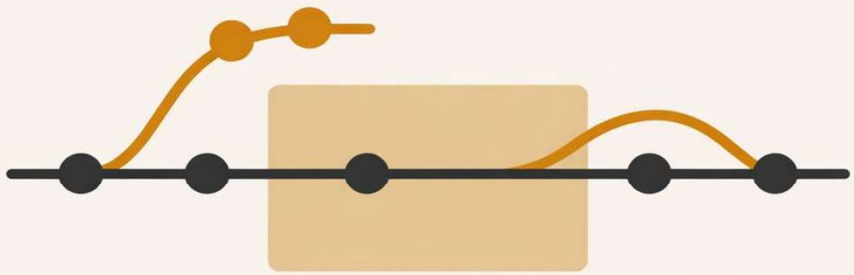
Trabajo Colaborativo

Permite trabajo simultáneo y control de conflictos, mejorando flujo y productividad.

Reversión Sencilla

Deshacer errores es rápido y seguro; reportes citan hasta 60% menos incidencias por error humano.

Conceptos Clave de Git



Repositorio

Es el lugar donde se almacena el código y su historial de cambios, facilitando trabajo ordenado y colaborativo.

Commit

Registro de un conjunto de cambios: permite volver atrás, auditar decisiones y compartir avances con el equipo.

Branch y Merge

Una rama aísla experimentos; el merge integra el trabajo a la rama principal manteniendo el historial coherente.

"Git es como una máquina del tiempo para tu código: te permite retroceder, revisar y avanzar en cada versión, asegurando que nunca pierdas el rumbo."

↑

Introducción práctica a Git: configuración y primeros pasos

Setup inicial y Git básico

01

Instalar Git

Descargar e instalar Git desde el sitio oficial según el sistema operativo.

02

Configurar Identidad

Usar 'git config --global user.name' y 'git config --global user.email'. Una buena configuración reduce problemas en colaboración y seguimiento en un 30%.

03

Ajustar Preferencias

Configurar opciones como el editor predeterminado y el formato de fin de línea.

04

Crear Repositorio Local

Inicializar con 'git init' y comprender la estructura básica de carpetas y archivos.

Diferencia entre Git y GitHub

Git

Sistema de control de versiones local que permite gestionar cambios en proyectos de manera individual y offline.

GitHub

Plataforma remota que hospeda repositorios, facilitando la colaboración y el trabajo en equipo en línea.

Funciones Clave

Git: seguimiento de versiones, revertir cambios.
GitHub: colaboración, revisión de código, gestión de issues.

Relación

Git se usa para gestionar versiones; GitHub potencia la colaboración y el acceso remoto a los repositorios.

Demostración en vivo: comandos Git

01

git init

Inicializa un repositorio vacío. Ej.: git init → Crea la carpeta .git.

02

git status

Muestra el estado del working tree y staging. Ej.: git status → Archivos no rastreados.

03

git add

Agrega cambios al staging. Ej.: git add archivo.txt → Listo para commit.

04

git commit

Guarda el historial del repositorio. Ej.: git commit -m Primer commit → Nuevo commit registrado.

Staging y commits como fotos

Staging area = zona de preparación

Aquí agrupas y revisas cambios antes de confirmar: eliges qué “entra en la foto”.

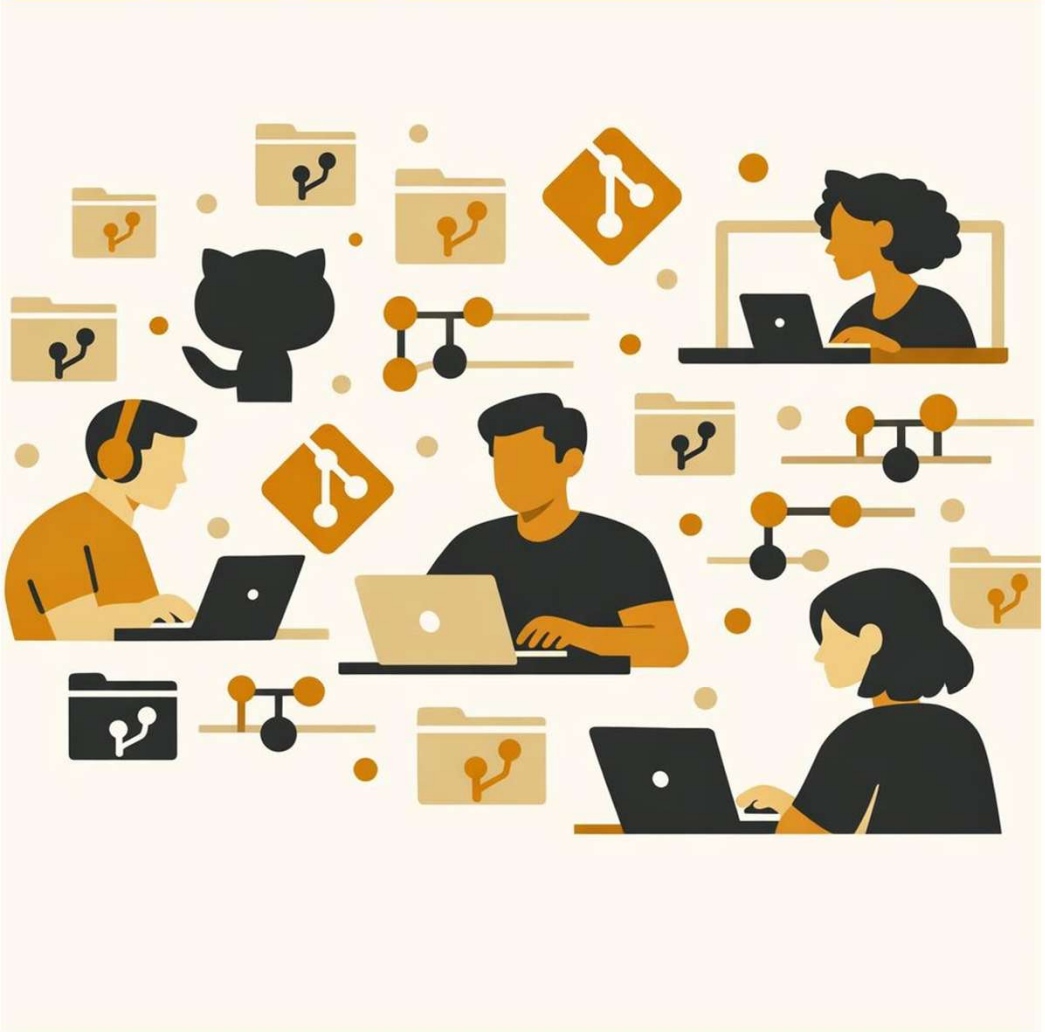
Commit = fotografía del estado

Captura el proyecto en un instante. Esta metáfora mejora el control de versiones y el flujo de desarrollo.

↑

Repositorios remotos y ramas en Git

¿Qué es GitHub y Git?



GitHub (plataforma)

Aloja repositorios Git y facilita trabajo remoto, revisiones y gestión de proyectos.

Relación con Git

Git es el control de versiones; GitHub es el servicio que lo pone en la nube para colaborar.

Impacto y escala

Más de **100 millones** de repositorios, estándar de la industria.

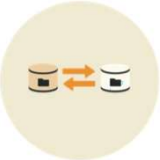
Diferencia entre repositorio local y remoto

Ubicación

El repositorio local se encuentra en la computadora del usuario, mientras que el remoto está alojado en un servidor accesible por internet.

Acceso

El local es privado y solo accesible por el usuario; el remoto permite colaboración y acceso simultáneo por varios usuarios.



Sincronización

Los cambios en el local deben sincronizarse manualmente con el remoto mediante comandos como 'push' y 'pull'.

Propósito

El local es para trabajo individual y pruebas; el remoto facilita trabajo en equipo y respaldo centralizado.

Conectar remoto y hacer push

01

Conectar remoto

git remote add origin <URL>
Esperado: sin salida; validar con
git remote -v

02

Preparar cambios

git status
git add .
Esperado: cambios “staged”

03

Confirmar commit

git commit -m “mensaje”
Esperado: resumen de archivos
y hash del commit

04

Subir al remoto

git push -u origin main
Esperado: “Writing objects…”
y rama actualizada

Concepto y utilidad de las ramas (branching)

Beneficios de Branching

Permite desarrollo paralelo, reduce conflictos de código y facilita la integración de nuevas funciones sin afectar la rama principal.

Casos de Uso con Branching

Corrección de errores, desarrollo de nuevas funcionalidades y experimentación controlada.

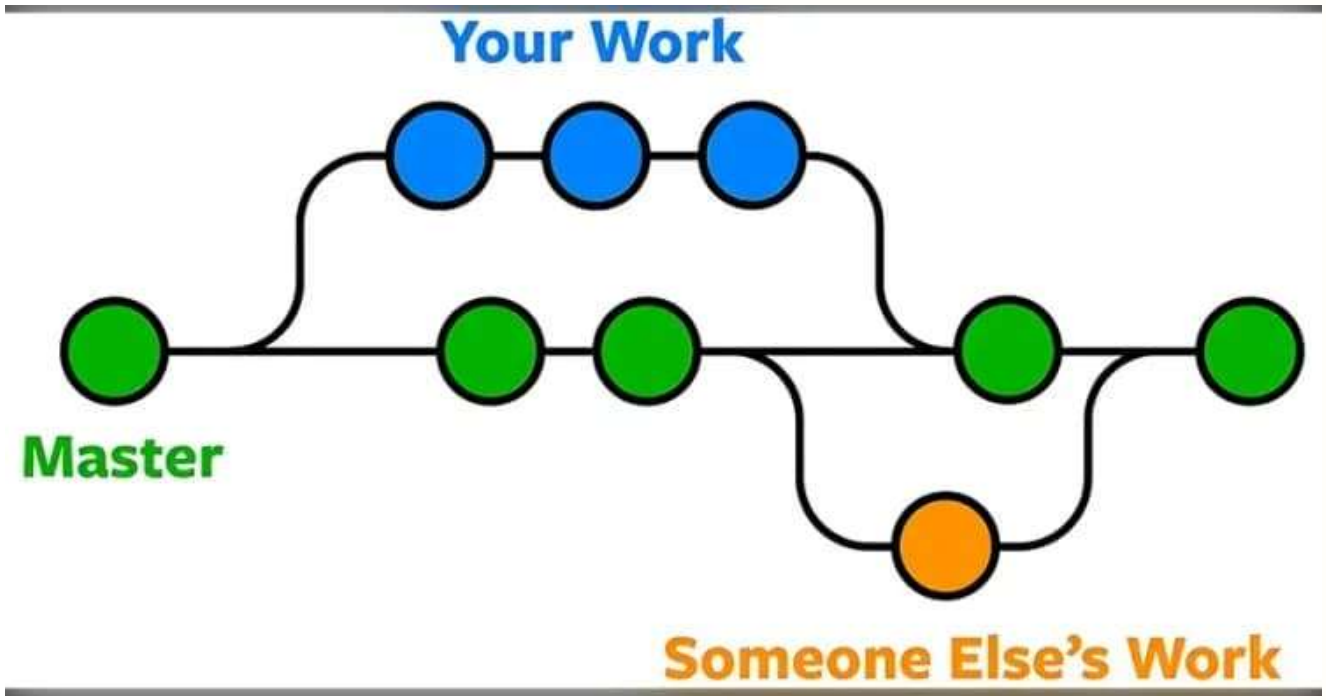
Limitaciones sin Branching

Mayor riesgo de conflictos, dificultad para revertir cambios y menor flexibilidad en el flujo de trabajo.

Casos de Uso sin Branching

Proyectos pequeños, tareas de mantenimiento simples y equipos con baja colaboración simultánea.

Concepto y utilidad de las ramas (branching)



Demostración: creación, cambio y fusión de ramas

Crear rama

El comando 'git branch' permite crear una nueva rama para desarrollar funcionalidades sin afectar la rama principal.

Cambiar de rama

'git checkout' se utiliza para moverse entre ramas, facilitando el trabajo en diferentes contextos del proyecto.

Fusionar ramas

'git merge' integra los cambios de una rama en otra, consolidando el trabajo realizado y manteniendo la integridad del código.

Evitar conflictos

Una fusión bien gestionada minimiza conflictos y asegura que el proyecto avance de manera ordenada y colaborativa.

"La rama main es como la versión estable en producción, mientras que las ramas secundarias son laboratorios de experimentación donde se prueba y mejora antes de lanzar a producción."

↑

Cierre, práctica guiada y primer repositorio

Mini práctica: crear repositorio

01

Inicializa el repo

Crea la carpeta, entra y ejecuta el comando de inicialización.

02

Añade archivos

Crea contenido base y prepara lo que irá al primer registro de cambios.

03

Haz el primer commit

Registra el estado inicial para tener un punto de partida reproducible.

04

Conecta y publica

Vincula el remoto y envía el historial para colaborar o respaldar.

Ejercicio guiado: flujo de ramas

01

Crear rama

Inicia una rama para trabajar sin afectar la principal.

02

Modificar archivos

Aplica cambios y verifica el estado del repositorio.

03

Commit cambios

Registra un avance claro con un mensaje descriptivo.

04

Fusionar rama

Integra el trabajo a la rama objetivo con merge.

05

Resolver conflictos

Elige cambios correctos, prueba y confirma el resultado.

Resultado esperado

100%

de los estudiantes completaron un repositorio funcional, demostrando la aplicación práctica de Git y control de versiones

Conclusión: Dominando Git

- En esta presentación exploramos los fundamentos del control de versiones, la importancia de Git y GitHub, y su aplicación práctica para gestionar proyectos de forma organizada y colaborativa.
- Con estas herramientas y conceptos, estás preparado para crear, modificar y fusionar repositorios, facilitando el trabajo en equipo y el manejo eficiente de cambios en tus proyectos.

GRACIAS

ADN UPN

VALORES QUE NOS IMPULSAN A TRANSFORMAR VIDAS

Hacemos lo correcto

Generamos el cambio

Trabajamos como ⇑ solo

Actuamos como dueños

Gestionamos con agilidad

Nos apasiona lo que hacemos